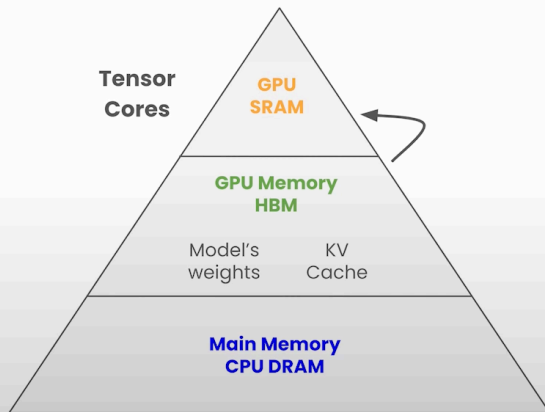


Quantization Schemes



Weight-Only Quantization (ex. W8A16)

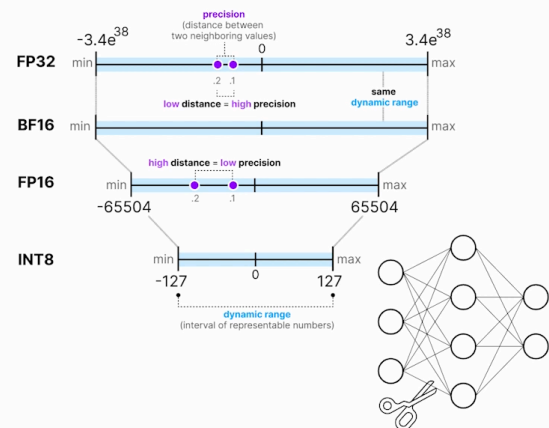
- Only Weights are quantized (ex. INT8)
- During inference, activations remain high-precision (BF16)
- At inference time, load the compressed weights from HBM to SRAM, “dequantize” the weights and compute with BF16 tensor cores
- Reduce data from HBM → SRAM

Weight & Activation Quantization (ex. W8A8)

- Both Weights & Activations are quantized (ex. INT8/INT4)
- At inference time, compute with tensor cores optimized for lower precision (FP8)
- Reduces data from HBM → SRAM & use tensor cores with more FLOPS

LLM Compression helps to reduce the model's size

- **Quantization:** A technique to compress LLMs by reducing numerical precision
 - LLMs are typically released in **BF16 (16-bit)**.
 - Quantization converts these into lower-bit formats (FP8, INT8, INT4).
 - Applied to weights & activations of model
- **Sparsification:** Removes unnecessary weights
 - Typically 2 of 4, where 2 out of every 4 values within a model weight tensor are set to 0
- **Both reduce memory footprint**



GPU Memory Hierarchy

Right next to tensor cores:
specialized hardware for
matrix multiplication

**GPU
SRAM**

smallest but
fastest

NVIDIA A100:
20 MB, 19 TB/s

**GPU Memory / VRAM
HBM**

smaller than host memory
but much faster

NVIDIA A100:
40 GB, 1.5 TB/s

**Host Machine's Main Memory
CPU DRAM**

large but slow

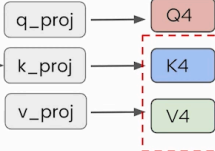
12 GB/s

KV Cache

Step 1

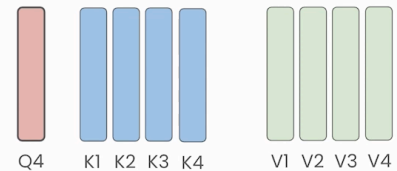
"The quick brown fox"

Vector
representation for
fox



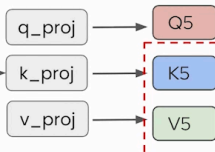
Save them
to KV Cache

Steps 2 & 3 (attention & output computation)



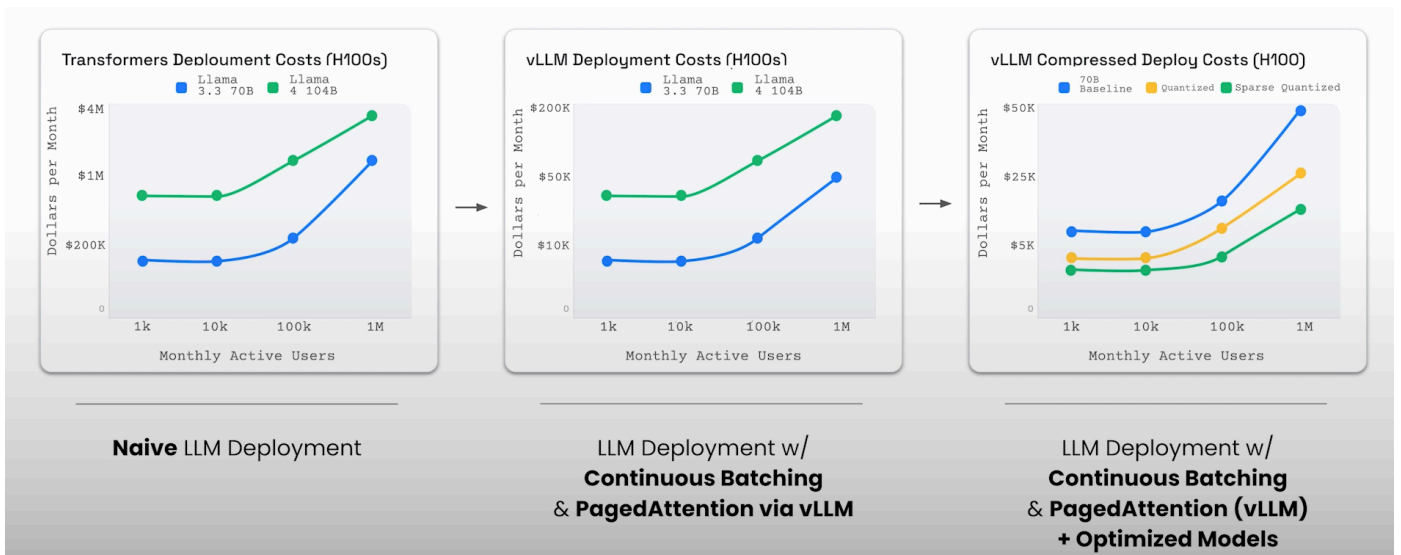
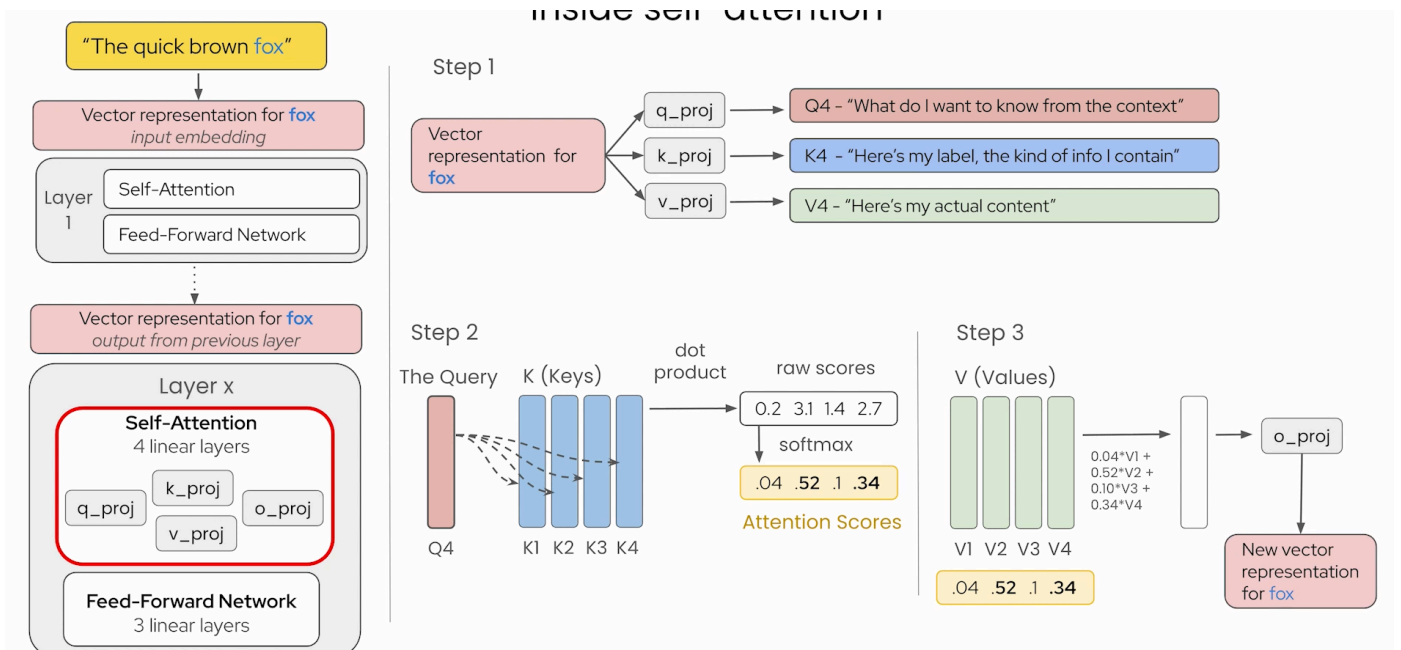
"The quick brown fox jumps"

Vector
representation for
jumps

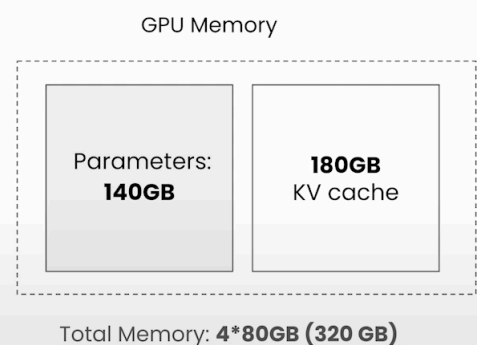


Append them to
the KV Cache





- **GPU memory:**
 - Model weights/parameters: fixed space
 - KV cache: grows with every token in every active request
- **Llama 3 70B parameters:**
 - **Weights: 140GB** (2 bytes per parameter) → at least 2x 80GB GPUs
 - In practice, deploy it on 4x 80GB GPUs (standard unit)
- **Llama 3 70B KV cache:**
 - One long-context request (32K tokens) needs **10GB in KV cache**
 - With 180GB, you can serve **18 long-context users in parallel**
- **Scaling users with naive serving setup:**
 - 2 or 3 parallel users



- **Accuracy Service Level Objectives (SLOs)**

- **Accuracy** must exceed usability thresholds to avoid
 - Hallucinations (inaccuracy or incorrect)
 - Off-brand responses

- **Inference Performance SLOs**

- **Latency** must not get in the way:
 - Time to first token (TTFT): the time taken to generate the first output token
 - Inter token latency (ITL): the average time between generating consecutive tokens in the output, excluding the first token
 - Request latency: the total time end to end
- **Throughput** must scale:
 - The average number of output tokens generated per second across all requests

Open LLM Leaderboard evaluation scores

Benchmark	Meta-Llama-3.1-7 0B-Instruct	Meta-Llama-3.1-7 0B-Instruct-FP8	Recovery
MMLU (5-shot)	83.83	83.73	99.88%
MMLU-cot (0-shot)	86.01	85.44	99.34%
ARC Challenge (0-shot)	93.26	92.92	99.64%
GSM-8K-cot (8-shot, strict-match)	94.92	94.54	99.60%
Hellaswag (10-shot)	86.75	86.64	99.87%
Winogrande (5-shot)	85.32	85.95	100.7%
TruthfulQA (0-shot, mc2)	60.68	60.84	100.2%
Average	84.40	84.29	99.88%

Example of Llama 3's Model Card

In practice, optimizing for any two hurts the third:

- ▶ High accuracy + low latency = **high cost**
- ▶ Low cost + high accuracy = **high latency**
- ▶ Low cost + low latency = **low accuracy**

Choosing the right model, performance targets, and hardware setup is complex; without clear measurements, it's hard to make informed decisions.



Compressing an LLM with LLM Compressor

1. Choosing a Model and Dataset

Models can be sourced from repositories like Hugging Face or your own model catalog

3. Choosing a Quantization Scheme

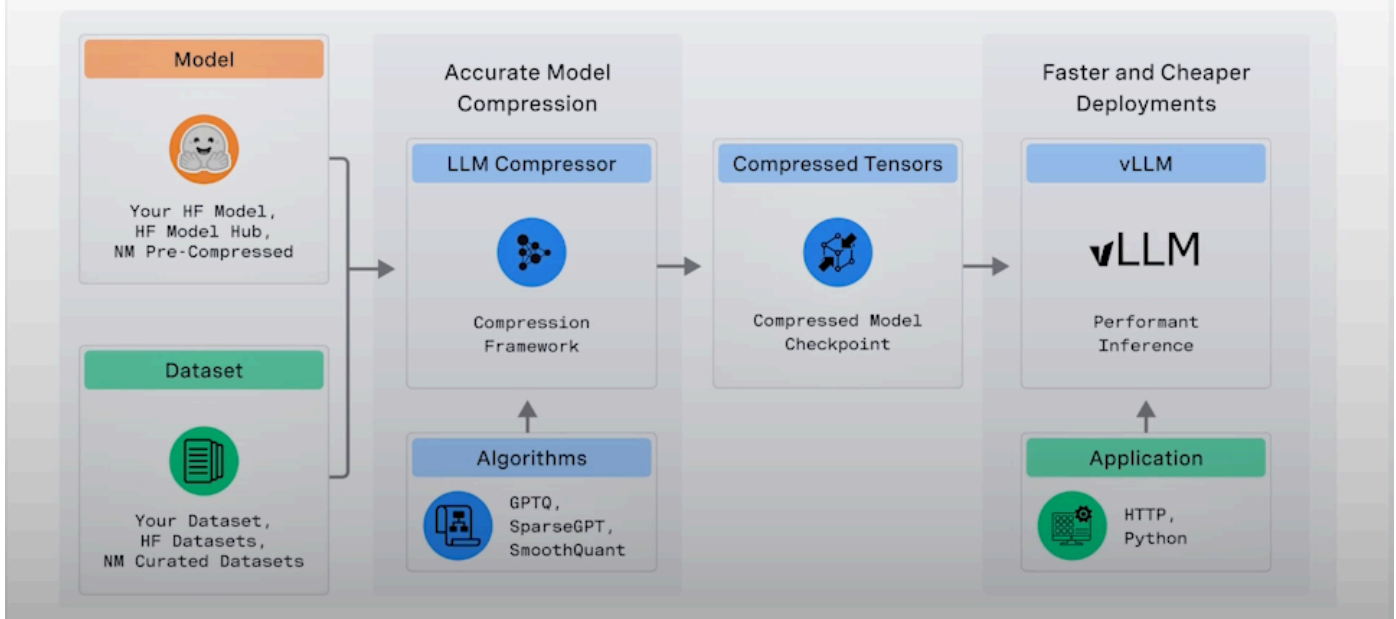
This means optimizing the model for latency, hardware, and throughput (ex. *W4A16* vs *INT8*)

2. Picking the Algorithm

Different ML algorithms can be used for LLM Compression (ex. *GPTQ* vs *AWQ*)

4. Inferencing with vLLM (Next Lesson)

vLLM directly supports various quantization for acceleration of AI models



PagedAttention

Seq
A

Prompt: "Artificial Intelligence is"
Completion: "the future of technology"

Block table

Physical block no.	# Filled slots
3	4
6	3

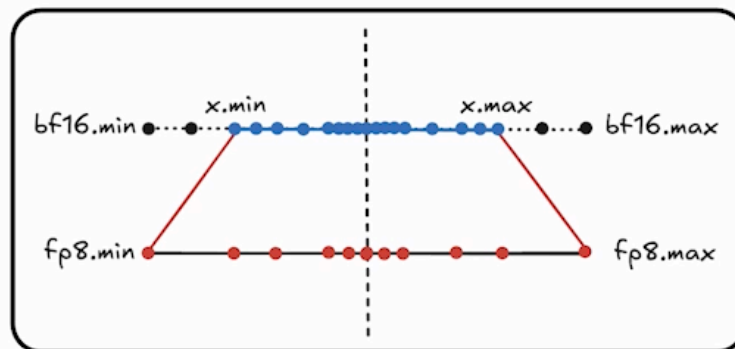
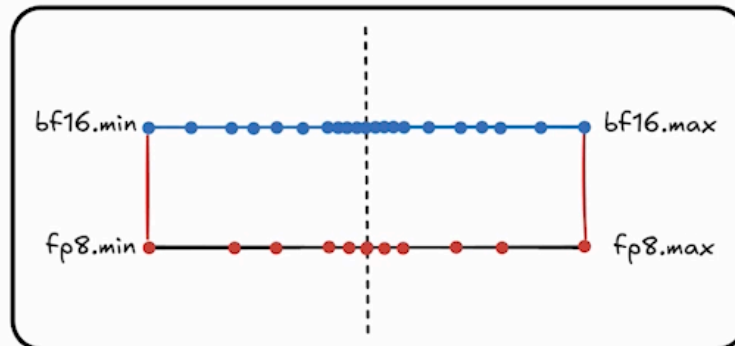
Physical KV cache blocks (GPU memory)

Block 0				
Block 1				
Block 2				
Block 3	Artificial	Intelligence	is	the
Block 4				
Block 5				
Block 6	future	of	technology	
Block 7				

Instead of storing KV cache as one large continuous block, it's broken up into **fixed-size blocks (pages)**

Picking the Quantization Algorithm

QuantizationModifier: Round to Nearest (RTN)



```
recipe = QuantizationModifier(  
    targets="Linear",           # layers to quantize  
    scheme="FP8",               # weight/activation scheme  
    ignore=["lm_head"],         # layers to ignore  
    kv_cache_scheme=None,       # kv cache quantization  
)
```

Picking the Quantization Algorithm

Access the latest compression research

Weights & Activations ▶ Round-to-nearest Fastest compression, baseline accuracy. Best for CPU/Mac (GGUF) & experimentation.	Weights only ▶ AWQ Best accuracy/speed balance for NVIDIA. Faster calibration than GPTQ, highly preferred for INT4. ▶ GPTQ Mature, widely supported, requires large VRAM for compression. Industry standard for compatibility.
Sparsity ▶ Sparse-GPT For extreme compression, but only for specific GPU's like NVIDIA H100 (for 2:4 pattern)	Transforms and Smoothing ▶ SmoothQuant Best for weight & activation quantization (INT8) ▶ SpinQuant & QuIP Can compress without needing a calibration dataset (low compute environment)

Typical use cases we need to plan for

"With NVIDIA H200, should I use a Llama 3.1 8B or 70B Instruct to create a customer service chatbot?"

Pre-deployment

Inform model selection, size and family

"How many servers do I need to keep my service running under maximum load?"

Cost & Capacity planning

Predict infrastructure needs under projected user loads.

"How much more traffic can Llama 3.1 8B Instruct FP8 handle over the baseline?"

Regression & A/B Testing

Track performance deltas triggered by model updates and performance alerts.

"What is the max RPS that my hardware can handle before my performance begins to degrade?"

Hardware evaluation

Compare latency and throughput across GPU/CPU instances